Developer's Guide

Where to find the code?
private GitHub with the latest updates under laurascrazythings, it is under public MIT license
https://github.com/laurascrazythings/CNRS-EM2C-MODELE-FILM-NANO-PARTICULE

Downloads to do:
Numpy,
Matplot
Ffmpeg
Mpiexec
Python in the same folder than mpiexec (which mpiexec and which python are the commands to make sure they are in the same folder)

Set up to adapt in Simu.py:
Maybe change python to python3 and maybe look at the folder you have to ensure it matches the location of the file.

Run Simu.py from the terminal, ensure you are using the right python, right path…

Goal of the simulator:
The goal of the simulation is to predict the layout of particles in a medium after a certain time given the mass flow rate, the medium, and the particles properties like tis radius, density and more.
It was specifically designed to predict the coating layout of TiO2 particles. A good explanation can be found in project 3A on teams.

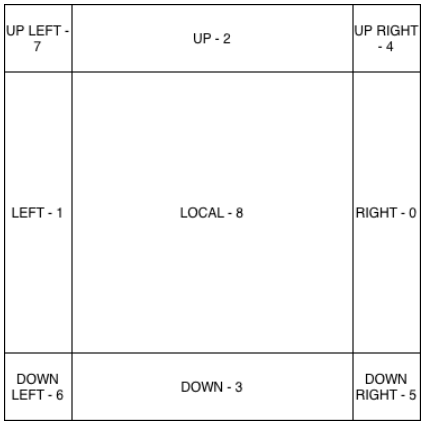
What was here before?
When first starting this project, students had worked on a simulation with agglomerating particles. Those files can be found under *CODE SOURCE.* They had a brownian motion simulator that would simulate agglomeration or collision depending on a coefficient. It is well commented, and inspired me for the code, even though 95% of the algorithms had to be rethought for efficiency and reliability: tunneling was happening, particles were skipping each other… However, It was a great start.

Thought Process:
This next section will explain step by step the coding process. Highlighting the different parts of the code.

The main concern was run time and accuracy. Being able to simulate millions of particles on a single machine with reliability and following physics law to ensure applicability to the real world.

To do so, the concept of message per interface also known as mpi was introduced to the project. MPI (links below)

is a downloadable library that allows you to separate the different cores of processors in your machine. It is usable in a super calculator. It allows you to treat cores as a computers and manage the communications between them.



In this numerical simulation, mpi is used to separate the domain like a cube in a rubik's cube. Once particles leave the cube, they will be sent to the according neighboring processor.

Step 1: Transporting a single particle across different processors in 2D.
The most part of the script code is moving particles across different processors. To ensure no particles would be lost,

ghost zones were create (image here)The color of the line represent the ownership of the line to a certain zone - numbered 0-8. Depending on the ownership the inequality signs will have an equal sign or not of the condition loops to ensure a particle can only be in one zone at one time for a designated processor. A particle can only be in one “local zone” at once but can be in ghost zones of the neighboring zones.

Step 2: Introduce collisions (links below)
After studying particles dynamics, collision functions were created as can be seen in *particle_interactions.py* . In the for loop, the first thing that is done is updating the velocities. Collision coding happens in 3 steps: *Broad_detect* which returns the particles within a certain distance d, *narrow_detect* which predicts if the particles will hit each other during the delta t and returns the time of collision and first collision. And *update_particles_collision* which returns the updates Attributes values after collision.

Step 3: Add Brownian motion (links below)
Brownian motion is approximating the effect of smaller particles (median like air or water) around the studied particles (here TiO2). In this simulation we are using Ornstein Uhlenbeck’s equation.

Step 4: Agglomeration
Particles can also be agglomerating, if their velocity isn’t great enough to bounce back (see Lennard Jones). Hence, another function with the same return than collision, called *update_particles_agglomeration* can be found in *particle_interactions.py*

Step 5: Spin
Starts with the center of gravity, stopped here for now

Step 6: User Interface
To make the simulator accessible, a user interface where the user gets to choose clearly their set up and see the progress as the code is running was required. The code for those can be found in : *Gui.py* for the user interface and *Progress_gui.py* for the progress bars.

Errors codes and meaning:

Example:
The position [66.68250616 99.99] for particule 145 in proc: 4 is different than in the master at t = 762
which is : [0. 99.99]
=> This error is due to a mismatch between one processor already merged with the master and the new processor being added. Happens 1/30 000 particles or so. I believe it is due to the particle accelerating, and hence not having time in the buffer to be updated in and hence transferred to the next processor, the particle was retrieved explaining the right position in the master. I wonder why the error is still showing…

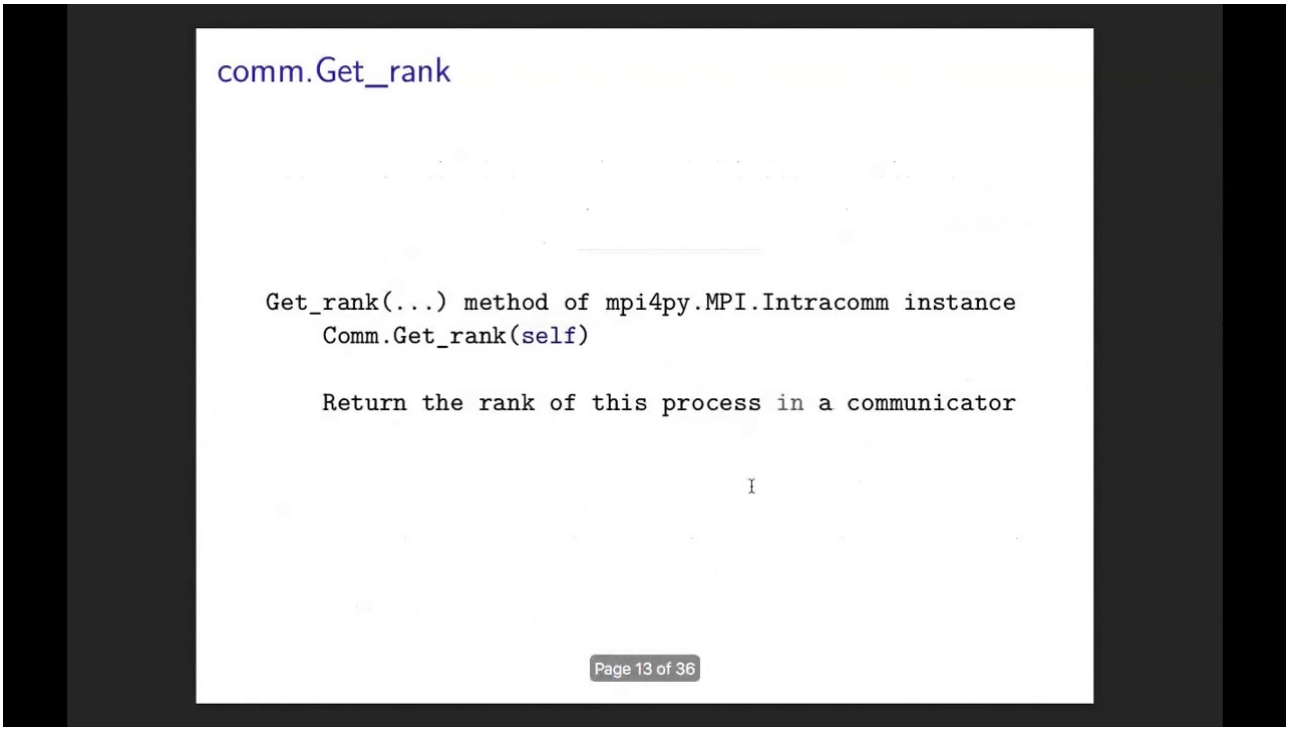
Missing indices: [] ... total: 0
Example values: []
=> If the number is anything but 0 it means the spawning zone was too small for the number of particles that needed to be generated. As 2 particles cannot be on top of one another. Try reducing the number of particles of increasing the domain.

What is next?
Spin!
3D

Journal with the videos (in French):
Jour 1: lecture sur la thermophorese, le mouvement brownien, lecture
https://www.techniques-ingenieur.fr/base-documentaire/sciences-fondamentales-th8/etats-de-la-matiere-42109210/physique-des-aerosols-af3612/phenomenes-de-thermophorese-photophorese-diffusionphorese-af3612niv10003.html
Jour 2:
Lecture plus pousee installation de git et du code creation d'un repository, mise a jour du code,
Il y a eu une update sur numba qui explique le bug
En nopython, le code tourne en 4min59 pour 15 secondes
Jour 3:
Comprehension du code meilleure, pourquoi ne pas tout transformer en fortran ? A terme, si on veut améliorer l'agorithmie, il faudrait : enlever les if, transformer les boucles for au possible; pour faire cela, on pourrait soit faire des matrices de mesh soit des matrices de taille particule. A voir si possible.
Plongee sur MPI

Distributed Computing with MPI

Part 1 - Basics



En enlevant les print, j'ai deja réduit de 45/300 le run time (15%) maintenant 1 secondes prend 16 secondes au lieu de 20, c'est deja cela. J'essaye de faire d'autres changements mineurs.
MPI foncitonne sur le pc, mise en place d'un code qui teste - fonctionne
Code ou une particule bouge d'un processeur a l'autre - en cours
Voici un code qui utilise du MPI
Pour

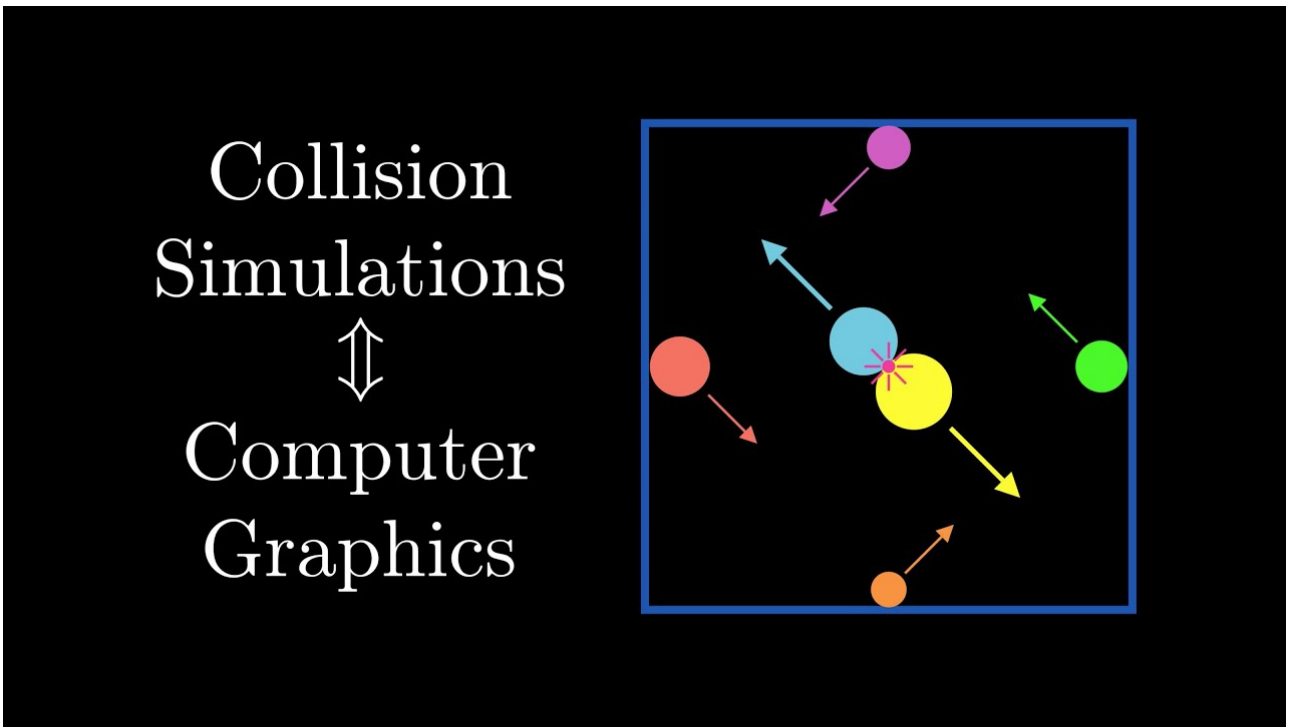
```
#I am wanting to code a mpi code that can send one particule from
one proc to another
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD #INIT the mpi
my_rank = comm.Get_rank()
my_size = comm.Get_size()

a = 0.0
b = 1.0
n = 1024
dest = 0
total = -1.0
h = (b - a) / n

local_n = n // my_size
local_a = a + my_rank * local_n * h;
local_b = local_a + local_n * h
# build local grid and values
x_local = np.linspace(local_a, local_b, local_n + 1 )
y_local = (x_local)**2

# local integral
```

```
integral = np.trapezoid(y_local, dx=h)
#add up the integrals
if my_rank == 0:
    total = integral
    for source in range (1, my_size):
        integral = comm.recv(source = source)
        print ("Processor", my_rank, "<-", source, ",", integral,
"\n")
        total = total + integral
else:
    print("Processor", my_rank, "->", dest, ",", integral, "\n")
    comm.send(integral, dest = 0)

#print the result
if(my_rank == 0) :
    print("With n = ", n, "Integral from ", a, "to ", b, "is equal
to ", total)

MPI.Finalize
```

Code MPI une particule vers un autre processeur - en cours

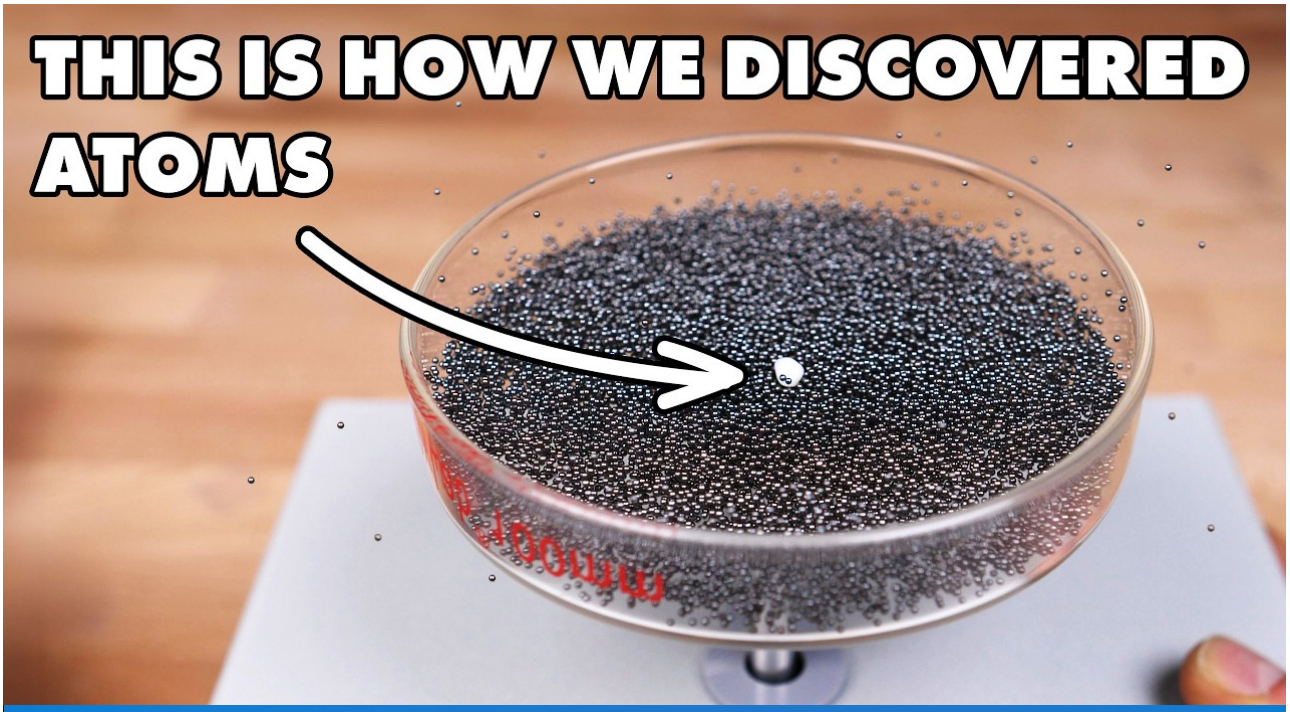
Lundi 8/12:
INSTALL OPENMPI
Pour installer open mpi, recup les droits admin sur les fichiers conda, créer un environnement anaconda en faisant un conda create envs « le nom »; reinstaller conda install openmpi mpi4py ; reinstaller tous les extras; activer le conda en faisant conda activate « le nom » et pour tester verifier que le which python et le which mpiexec sont dans le meme dossier.
Pour le faire tourner:
mpiexec -n 4 python CNRS-EM2C-MODELE-FILM-NANO-PARTICULE/CODE_2D_PARALLEL/

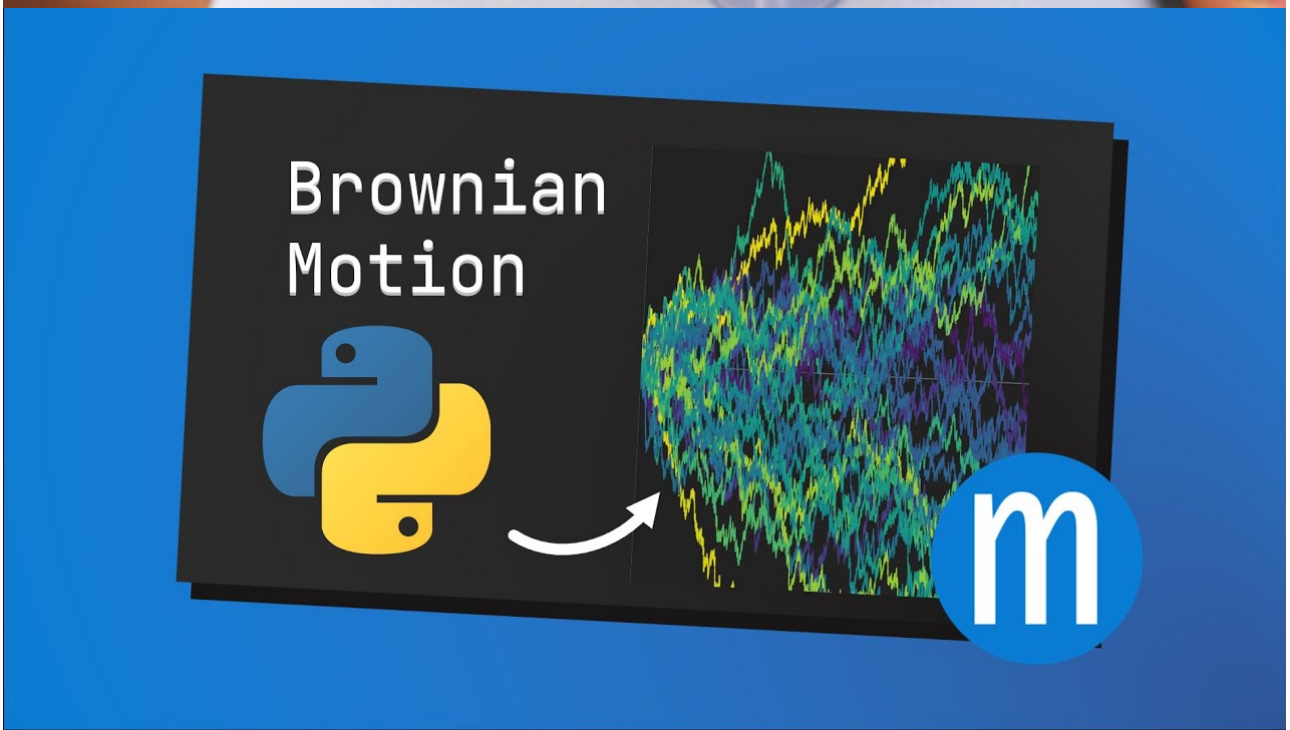
Mercredi :
Avancement sur le code (tout est commente)
Jeudi:n particules 1 dimension 1 direction !!!!!
Vendredi: it is plotting, going for 2 directions ca marche, c'est commit!!!!!!!



Mercredi :
J'ai creee une branche, le code marche en 1d avec condition périodique, je le transforme en 2d la 31/12 le code fonctionne en 2d sans soucis, les angles fonctionnent après debug et reorganisation du code
5/01: ajout progressif de particule,
6/01: ajout progressif de particule en calcul matriciel et invisible au depart
7/01: kd tree

Pour l'instant je fais de l'eulerien, donc du discret, ce qui fonctionne tres bien tant qu'il ny a pas de discontinuité, j'ai deux choix: check les collisions entre 2 dt ou réduire le dt pour que les collisions ne soient pas possibles entre 2 dt.
Je pense commencer par le changement de dt en fonction de la vitesse maximale et ensuite implementer la recherche continue si des erreurs se montrent
I changed the code so that instead of adding a specific number of particles every delta t, it is adding them every second, we can lower the every second to something bigger than dt or a multiple of it.
Cela ralentit enormement le code d'avoir un dt si raffine, je pense que la méthode de check intermédiaire est nécessaire .

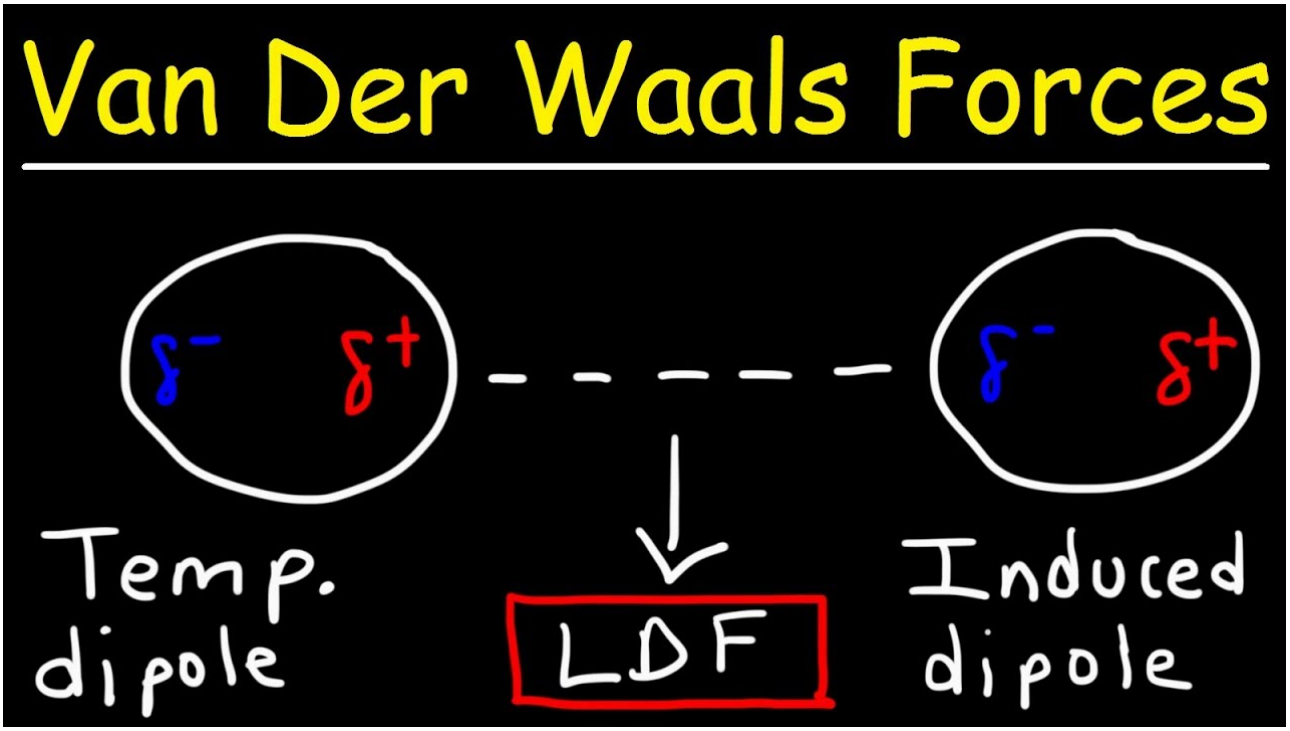


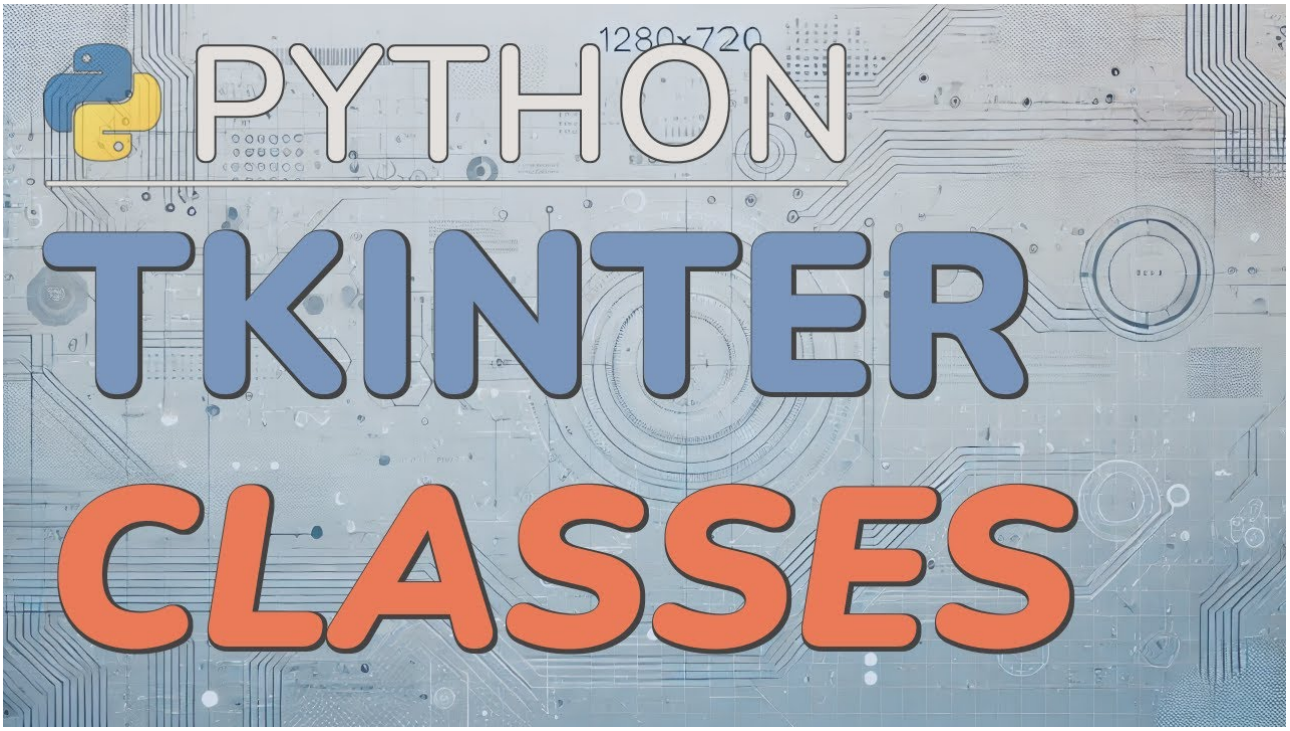


20/01: 2d collisions work, had to do a lot of exception coding, I am now trying to implement brownian motion

https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2024JB030445?af=R
Coefficient of restitution
Brownian motion was implemented, as well as the coefficient of restitution. That all works, patches were made on the transport conditions to ensure no particles could leave.

26/01: working on adhesion and especially the condition of adhesion,



Book on interactions between particles:
https://www.eng.uc.edu/~beaucag/Classes/AdvancedMaterialsThermodynamics/Books/Jacob N. Israelachvili - Intermolecular and Surface Forces, Third Edition_ Revised Third Edition-Academic Press (2011).pdf

Make a gui: